

Storybook as Your Component Contract

So...How many of you have had a ticket kicked back because of a display issue?

Quick show of hands

How many of you have heard something like...

Make it look like it does in Figma

And the only thing you can say...

"It looked right on my machine..."

Or...maybe even worse...

The docs say one thing...

But the component that you've been using...

Looks completely...different

The problem isn't that we don't document our components

The problem...

Is that our documentation...

Can lie

Storybook can help

But, Storybook only works when you stop testing it like documentation...

And start treating it...

Like a contract

The biggest problem we've probably all experienced...

Drift

Your component design has evolved...

But your design docs didn't

Or...

Maybe there was some edgecase...

Something that had to be solved...

Right then

New props were added...

The names probably even made sense at the time...

But now...no one is really sure what they do

They kind of do 2 or 3 things across the component...

But they still aren't documented anywhere

Or maybe your team needed something that already existed...

But...it needed to do something else...too

Now...instead of having one Email Form...

You've got two.

Or maybe your brand has changed

And now none of your components have the right...anything

Screenshots are stale

And docs are referencing things that don't exist anymore

Drift is inevitable...unless something enforces the truth

Anything that isn't tied to the code...**will**...drift from away from the code

Fundamentals (done right)

Most teams have Storybook...but few teams are using it correctly

We've probably all done this

"Did my button work?"...Click...

and then we open the console and look for the console log.

What have you tested here?

We tested that a HTML button has a click handler

What is that click supposed to do?

In reality...probably not log to our console...right?

Storybook is meant to show the **state**...of our component

Storybook be showing us...what are component looks like
In this example ...
We can see what it looks like
Unless this button has a single purpose...
Unless this button always opens a modal...
Or a dialog...
And never does anything else...
That click doesn't help us here.
It feels like we tested something...
But we didn't test anything meaningful
We have more important things to be testing...

This displays the same button
It has the same label
But now...
We've made it easy to test the **state**
Is it disabled?
No
So we know what styles...**should**...be displayed
And we can check those
Create another story just like that one ...
Except that we flip the flag
Did our styles update?
No?
Now we have something we can look into
Yes? Awesome! We win!
Wait...only **some** of the styles are present?
You see...
Now we get actual feedback from this...
Now we know what work, if any...remains
If it can vary... it should be an arg
Can you still have the click handler?
Sure!
We can all use an easy win sometimes
But!
Does it always do the same thing?
No?
Make it an arg

The place where we go wrong...
Too often...
Is that we hardcode our stories
In the example up here
We were declarative

Because of the args...
Our test is *controllable*
Because we are declarative...
And...
Because we are controllable...
We are...*testable*

Controls make exploration possible
Controls turn your component into an...
API Explorer

This helps designers...
This helps QA...
And this helps new devs...
Now everyone can understand the system
Now everyone can understand the components role

Autodocs!
This is where we start to see the power...

What is autodocs?
Exactly what it sounds like.

With autodocs...
We get some very quick wins

We get what props this component takes in
What it's called...
What it's for...
Does it have a default?
This is automatic power
This is a huge win
Have you used Swagger for endpoint documentation?
Same idea
Easy setup, instant docs

You get examples
You open your button component docs
You see your props
And you see your different buttons
If you created a story for it...
If you had different args...
It's displayed here..

With a title so you know what you're looking at
Different styles...
Different states...
One quick reference

You changed a story?
No problem...
The docs updated
You added a story?
The docs updated
New prop?
You guessed it...the docs updated

Autodocs works because it's generated from the same source as your component
No duplication
No drift
This is no longer just documentation
This...*is*...your contract

If a prop can vary...
It should be in our args
Play functions are almost like mini end 2 end tests...
Scoped directly to our components
Instead of testing our color...
Or just verifying it rendered...
We test our interactions...
We test our behavior
This is the answer to...
"Can my user do the thing?"

When I first started using play functions...
I thought—"Great! I can test everything here"
Clicks...
Behavior...
Even styles...
So i tried asserting colors
And i realized...
I just made the same mistake...I warned about earlier
I was testing the implementation...not the outcome
Here's what that looks like...

Now...we're not just showing the component...
We're verifying that it works
We've given our *contract*...
a way to enforce behavior...
Inside Storybook
Our contract...
Just got stronger
This isn't documentation anymore...
This is enforcement
Remember
If it breaks in Storybook
It never ships.

But behavior is only part of our contract

What about how it looks?

We've defined our state...
We've locked in our behavior
Now...we protect them
This is where visual regression testing comes in
Every story now becomes a snapshot...
Every snapshot...a baseline
And every change...becomes visible

Using a tool like Storybook test runner...

We gain the ability to *know* when something changes
Not guess...
Not hope...
Know

It turns our stories...into tests
Not just for behavior...
But for appearance
And this is where those things come back in...

Colors
Spacing
Layout
Rendering

Not as implementation details...
But as visible...outcomes
This is how you catch change...
Before your users do
This enforces your standards
Because that's what a contract does

Now...

What happens when the *system itself* changes?

What happens when your brand updates...
Your spacing shifts...
Your colors change...

This is where design tokens come in

Design tokens define the *source of truth*

Your colors
Your spacing
Your typography

But tokens alone... aren't enough

Because tokens define values...

They don't guarantee how those values are used

This is where everything comes together

Design tokens define the system...
Storybook defines the contract

And visual testing...

Enforces it

Now when a token changes...

Your components update

Your stories update

Your snapshots update

And if something breaks...

You know immediately

No more guessing which components are affected

No more hunting through the app

No more "I think this might be safe..."

You *see* it

You update a primary color token

Every button changes

Every alert changes

Every link changes

And in seconds...

You know exactly what that impact is

This is what a system looks like

Tokens define the rules

Storybook defines the expectations

And your tests... enforce them

Tokens define the system...

Storybook proves the system works.

Now...You're ready for a real challenge

Storybook fails when it becomes the...side project

Something that we...mean...to update

Something that we...plan...to update

But we don't

You have to start by making it the default

Link it in your PR's

Leave comments asking if people have updated storybook

Bring it up and use it in design reviews

Make not using storybook...

Harder than using it

Adoption doesn't come from rules, though...

It comes from usefulness

Make the cases real

You don't need all of the edgecases

No hypotheticals...

You need what is really happening in *your* app

Make states visible

It can be disabled?

It should be in a story.

If it can vary...

It should be visible

Every meaningful state...

Should be represented

If your stories aren't useful

They will be ignored

And remember:

Storybook isn't for frontend devs...

It's for designers

It's for QA

It's for product

It's for anyone...

Who needs to trust the UI

You started with a problem...

Things didn't match

Docs said one thing...but code said another

And we all had the same fallback...

"It worked on my machine"

The problem isn't going away...

Unless we change how we define our components

We have to stop treating Storybook like Documentation

We need to start treating Storybook like a contract

Documentation tells you what should happen

A contract guarantees what does happen

Because when your components have contracts

There's no more guessing

There's no more drift

There's just

Clarity

And that's how your system becomes something people can trust

If it breaks in Storybook... it never ships.